投机解码（Speculative Decoding）：原理、代码与实现细节

0. 这份文档怎么读

本文档讲清四件事：**投机解码在做什么、我们的代码怎么实现的、每个输入/输出张量到底代表什么、每个 PyTorch 操作在算什么。**全文分三层，建议按顺序读：

- **第一层（第 1 节）原理**：为什么自回归解码慢、投机解码凭什么省时间、它为什么"不改变输出"。
- **第二层（第 2 节）数据流**：一次生成从 prompt 到输出 token 的全景，配一个真实轮次的逐步演算与真实数字。
- **第三层（第 3–5 节）逐行代码**：`spec_dec.py`（116 行）、引擎张量细节、`spec_cache.py`（65 行）。
- 之后是**正确性证据**（第 6 节，含一个真实的"差一个 token"案例及其归因）、**常见误解 Q&A**（第 7 节）、**速查表**（第 8 节）。

行号约定：`spec_dec.py` 共 116 行、`model_self.py` 共 319 行、`spec_cache.py` 共 65 行；行号对应仓库 `main` 分支 `000cc9b`。所有数字都来自本项目实测（台账 `docs/data_ledger.md`），没有估算值被写成实测值。

记号：形状写成 `(B, S, H)` 形式 —— `B`=batch（恒为 1）、`S`=序列长度、`H`=hidden 或 head 维；`V`=词表大小 151936；`T`=KV cache 的 token 维长度。`L/pre` 都表示"本轮开始前 cache 里已有的 token 数"。

1. 第一层：投机解码到底做了什么

1.1 自回归解码为什么慢

生成第 t 个 token，必须完整跑一遍模型（28 层、全部权重），但只产出 **1 个 token**。在 batch=1 的 decode 阶段：

- 算术量极小：每个 token 大约 2 × 参数量 次浮点运算（1.5B 模型约 3 GFLOP）；
- 但要把**全部权重**从显存读一遍：bf16 的 1.5B 模型约 **3.1 GB**。

于是瓶颈不是算力，而是显存带宽 —— 这叫 **memory-bound**。粗略地说，若有效带宽 ~200 GB/s，单 token 的时间地板就是 3.1 GB / 200 GB/s ≈ 15 ms，而算力在这段时间里大部分闲置。**这就是"一次前向只产 1 个 token"的浪费所在，也是所有 decode 加速手段的出发点。**

1.2 核心想法：一个猜、一个批改

投机解码引入两个模型：

角色	我们的配置	职责
draft（草稿模型）	Qwen2.5-**0.5B**-Instruct	便宜地**猜**接下来 k 个 token
target（目标模型）	Qwen2.5-**1.5B**-Instruct	**一次块前向**同时"批改"这 k 个猜测

一轮的流程：

1. draft 自回归猜出 `d = [d1, d2, d3, d4]`（k=4，每猜一个跑一次 0.5B 前向）；
2. target 把 **4 个猜测一次性**喂进去（一次前向，位置 pre…pre+3），得到 4 行 logits；
3. 从 `d1` 开始逐个比对 target 的 argmax：相同就**接受**，遇到第一个不同就用 target 自己的 token **替换**，并把后面所有猜测丢弃；
4. 继续下一轮。

关键点：target 在这一轮里只做了一次前向，却可能产出 4 个 token（全部接受时）。省下的正是"每产 1 个 token 就要读一遍全部权重"这件事。

1.3 为什么它"不改变输出"（greedy 情形）

我们协议冻结为 **greedy + batch=1 + 无 bonus**，这个前提下可以严格论证输出不变：

- greedy 下 target 对某个位置唯一"正确"的答案就是它自己的 `argmax`；
- 候选 `d_j` 被接受的条件是 argmax(target 在位置 j 的输出) == d_j，也就是说"**如果让 target 自己生成，它也会生成 d_j**"；
- 出现第一个不匹配时，r = argmax(target 在该位置输出) 就是 target 自己会生成的 token；
- 因此每一轮产出的 token 序列，与 target 单独跑贪心解码**逐 token 相同。**

这就是为什么我们的正确性门（B6）用 **"逐 token id 完全相等"** 来判定，而不是比文本 —— 文本可能因解码/空白而骗人，token id 不会。

必须说清的边界：以上是 **greedy** 的结论。采样版本（temperature>0）不能用"精确相等"判定，而要用概率比接受（以 `min(1, p_target/p_draft)` 接受）并在拒绝时从修正分布重采样；那一套我们**没有实现**，协议里明确冻结为 greedy。答辩时若被问"采样怎么办"，答案就是这句话。

1.4 收益从哪来（成本结构）

一轮的成本与产出：

项	次数	说明
draft 前向	k 次（每候选 1 次）	0.5B 小模型，单 token

项	次数	说明
target 块前向	**1 次**	一次吃 k 个 token
target 单步重算	0 或 1 次	仅错配轮，用来算替换 token
产出	`m+1`（错配）或 `k`（全接受）	`m` = 被确认的候选个数

对比 AR：AR 要产出 `m+1` 个 token 就得跑 `m+1` 次 target 前向。投机解码把它压成 **1 次块前向 +（可能）1 次单步。**

这里有个容易误会的点：块前向的**算术量**约等于 k 个 token 的量（并不省算力），省的是**权重从显存读入的次数** —— 4 个 token 共享同一次权重读取。在 memory-bound 的 decode 阶段，这正是收益来源。

1.5 什么时候不划算

- **接受率 p 越低，收益越薄**：每轮产出少，而 target 前向次数不降（错配还要多一次单步重算）；
- **draft 前向本身要成本**：k 次 0.5B 前向在 target 很快时可能吃掉全部收益；
- **rollback 有固定开销**：cache 裁剪 + 替换 token 重算；
- **上下文变长**会让 e2e 里的 prefill 占比、以及每次 attention 的 KV 读取量都变化。

于是就有了本项目的两个研究问题：**RQ1**（上下文长度如何影响加速比）、**RQ2**（接受收益与验证开销各占多少）。第 6 节末尾给出的实测例子里，接受率 ~40% 时 target 前向只从 16 次降到 13 次 —— 收益已经很薄，这不是巧合。

---

2. 第二层：一次生成的全景数据流

2.1 三个核心对象

① 模型与张量

模型	hidden	heads	kv_heads	head_dim	layers	vocab
target 1.5B	1536	12	2	128	28	151936
draft 0.5B	896	14	2	64	24	151936

`heads / kv_heads = 6`（1.5B）和 `7`（0.5B），即 **GQA**：K/V 只有 2 个头，被复制到全部 heads（`kv_repeat`）。

② **logits**：`_run(..., need_logits=True)` 返回 `(1, S, 151936)`，dtype = 计算 dtype（GPU 上是 bf16，CPU 上是 fp32）。因为**权重绑定**（tied embeddings），它就是

```
logits = x @ embed_tokens.weight.T
```

没有独立的 `lm_head`。

③ **KV cache**：`list[dict]`，**每层一个 dict**：

```
base 档: {"k": (1, 2, T, 128), "v": (1, 2, T, 128)}
v_fp8 档: {"k": (1, 2, T, 128) bf16,     # K 不量化
        "v": (1, 2, T, 128) float8_e4m3fn,  # V 量化成 FP8
        "v_scale": (1, 2, T, 1) fp32,       # 每组一个 scale
              "fmt": "v_fp8"}
```

T 永远在 dim=2，所以"cache 里有多少个 token"就是 `cache[0]["k"].shape[2]`。全流程只有这一个长度来源 —— 不可能出现"逻辑位置"和"实际 cache"不一致。

2.2 流程（文字图）

```
prompt ids (1,S)
  | embed_tokens              查表 -> (1,S,hidden)
    v
[prefill] target._run(x, cache=None)  -> logits(1,S,V) + cache(len=S)
[prefill] draft ._run(x, cache=None)  -> logits(1,S,V) + cache(len=S)
    |
  | saved_next = tgt_lg[:, -1:]     (1,1,V)  <- 判 d1 的人
    v
+------------------------- 每轮循环（while len(gen) < max_new_tokens） -------------------------+
|  pre = len(tgt_cache)                                                                        |
| (1) _propose: draft 猜 k 个 -> cands=[d1..dkp]（draft cache 推进到 pre+kp）                  |
| (2) _verify : target 一次块前向吃 kp 个 -> rows (kp,V)（tgt cache 推进到 pre+kp）            |
| (3) 比对   : g = [saved_next, rows[0..kp-2]] 的 argmax，与 cands 求最长相等前缀 m           |
| (4) 分支                                                                                     |
|   m < kp（错配）: 产出 cands[:m] + [target 的 r]                                             |
|             tgt cache: trim 到 pre+m -> 再 _step(r) 写入 -> 长度 pre+m+1                     |
|             drf cache: trim 到 pre+m -> 再 _step(r) 写入 -> 长度 pre+m+1                     |
|   m == kp（全接受）: 产出 cands（kp 个）                                                     |
|             tgt cache: 直接用块前向的结果（长度 pre+kp，内容已经全对）                       |
|             drf cache: 裁到 pre+kp（幂等）；saved_next = rows[kp-1]                          |
| (5) 追加 gen，更新统计，遇 EOS 则结束                                                        |
+----------------------------------------------------------------------------------------------+
    v
gen（新生成的 token 列表） + stats
```

2.3 一个真实轮次的逐步演算

用我们 B5 smoke 的**真实首轮**（prompt 12 token，k=4，m=1 的错配轮）：

步骤	调用	涉及张量 / 长度	说明
起点	prefill ×2	`tgt_cache=12`、`drf_cache=12`；`saved_next (1,1,V)`	判 d1 的人已就位（位置 11 的 logits）
①	`_propose`	`cands=[d1,d2,d3,d4]`；drf 12→16；`prop_lg (1,1,V)`	draft 跑 **4 次**单 token 前向
②	`_verify`	喂 `(1,4)`；tgt 12→16；`rows (4,V)`	target 只跑 **1 次**
③	判决	`g=[saved_next,rows[0],rows[1],rows[2]]` 各取 argmax	**`rows[3]` 不参与判定**
③	匹配	假设 `g[0]==d1`、`g[1]!=d2` □ `m=1`	最长相等前缀
④	替换	`r = g[1]`（target 自己的 token）	与 `d2` 必然不同
④	产出	`emitted=[d1, r]`	本轮产出 **2** 个 token
④	裁剪	tgt trim(16 → 13 = pre+m)，丢掉 d2/d3/d4 的 K/V	**不能改写，只能裁掉重算**
④	重算	`_step(target, r)` → tgt=14；`_step(draft, r)` → drf=14	两个模型都必须走同一 token 序列
结果	—	`accepted += 1, proposed += 4, rounds=1`	target 前向总计 **2 次**（1 块 + 1 单步）

2.4 全流程的真实数字（B5 smoke，两档 KV 完全一致）

```
round0 Kp=4 m=1 pre=12 → tgt=drf=14   (错配: pre+m+1 = 12+1+1)
round1 Kp=4 m=0 pre=14 → tgt=drf=15   (错配: 14+0+1)
round2 Kp=4 m=0 pre=15 → tgt=drf=16   (错配: 15+0+1)
round3 Kp=4 m=1 pre=16 → tgt=drf=18   (错配: 16+1+1)
round4 Kp=4 m=4 pre=18 → tgt=drf=22   (全接受: pre+kp = 18+4)
round5 Kp=4 m=4 pre=22 → tgt=drf=26   (全接受: 22+4)
round6 Kp=2 m=0 pre=26 → tgt=drf=27   (预算只剩 2 □ k_eff=2)
round7 Kp=1 m=1 pre=27 → tgt=drf=28   (预算只剩 1 □ k_eff=1，全接受即结束)
```

统计：`rounds 8 / proposed 27 / accepted 11 / mismatch 5 / all_accept 3 / target_block_calls 8 / draft_calls 27, gen_len = 16`。

三个可从这张表直接验证的恒等式：

- **cache 不变量**：每轮 `tgt_len == drf_len == pre + (m+1 if m<kp else kp)`；
- **恒等式**：`gen_len == accepted + mismatch_rounds`（16 = 11 + 5）—— 因为每个错配轮恰好多产出 1 个 target token，每个全接受轮产出 `kp`（其 `kp` 个已全部计入 accepted）；
- **计数口径**：`target_block_calls == rounds`（每轮 1 次块前向）、`draft_calls == proposed`（每候选 1 次 draft 前向）。

这次生成的实际成本：**16 个 token，用了 13 次 target 前向**（8 次块前向 + 5 次单步重算）+ 27 次 draft 前向。对比 AR 的 16 次 target 前向 —— 接受率 11/27 ≈ 40.7% 时，收益已经很薄。**这就是 RQ2 要量化的东西。**

---

3. 第三层：逐行讲 `spec_dec.py`

3.1 模块头与常量（第 1–8 行）

```
1   from __future__ import annotations
3   import torch
5   from . import spec_cache
7   K = 4
8   FMT = {"base_bf16": "fp16", "v_fp8": "v_fp8"}
```

- 第 1 行：只是让类型标注延迟求值，运行期无影响。
- 第 5 行：导入用户实现的 cache 工具模块（`cache_length` / `trim_cache` / `check_consistent`）。
- 第 7 行 `K = 4`：每轮 draft 提议的候选数量上界，**协议冻结值**，不是可随手调的参数（调它要改协议）。
- 第 8 行 `FMT`：把**实验条件名**映射到**引擎内部的缓存格式名**。注意 `"fp16"` 在引擎里的含义是"不做任何量化的 base 缓存"，其实际 dtype 是计算 dtype（bf16 或 fp32）——**这是历史命名，报告里绝不能写成 fp16**，否则与协议 Amendment v7.2 的"Compute=BF16"冲突。

3.2 `_fmt`（第 11–12 行）

```
11  def _fmt(kv):
12      return FMT[kv]
```

输入档名字符串，输出引擎格式名。**故意不写 `.get(kv, "fp16")` 这种默认值**：档名拼错时立刻 `KeyError`，而不是悄悄按 base 档跑完一整轮实验、污染数据。这是"尽早失败"。

3.3 `_prefill`（第 15–18 行）—— 建立初始 cache

```
15  def _prefill(model, ids):
16      with torch.inference_mode():
17          lg, cache = model._run(model.embed_tokens(ids), None, need_logits=True)
18      return lg, cache
```

输入

- `ids`：`torch.LongTensor (1, S)`，`S = len(prompt)`，设备必须与模型一致。

内部每个 PyTorch 操作

- `model.embed_tokens(ids)`：`Embedding.forward` 就是 `return self.weight[token_ids]`，即**高级索引查表** → `(1, S, hidden)`。它不参与训练，只是把 token id 变成向量。
- `model._run(x, cache_list=None, need_logits=True)`：
- `start_pos = 0`（因为没有 cache）；
- `Build_Rope_cache(head_dim, seq_len=S, theta=1e6, device, dtype, start_pos=0)` 生成 `cos/sin`，形状 `(1, 1, S, head_dim)`；
- 逐层：`input_layernorm` → `self_attn` → 残差 → `post_attention_layernorm` → `MLP` → 残差；
- `cache=None` 时 attention 走"无缓存分支"：`torch.triu(ones(S,S), diagonal=1)` 生成上三角 bool mask → `scores.masked_fill(mask, -inf)` → softmax → `attn @ v`；
- `need_logits=True` 时最后算 `logits = x @ self.embed_tokens.weight.T` → `(1, S, 151936)`。
- `torch.inference_mode()`：不建 autograd 图、不做张量版本计数，省显存与时间。副作用是产生的张量是"inference tensor"，后续 `torch.cat` 结果同理，对本流程无影响。

输出：`lg (1, S, 151936)`（下一步只用最后一个位置，但整段保留便于诊断）+ `cache`（长度 = S）。

为什么这样设计：这是**唯一**直接读 `cache[0]["k"].shape[2]` 之外不依赖外部位置信息的地方 —— 之后所有位置都从 cache 长度推导，从根上避免 off-by-one。

3.4 `_propose`（第 21–33 行）—— draft 猜 k 个

```
21  def _propose(draft, cache, lg, k_eff, eos_id):
22      cands = []
23      with torch.inference_mode():
24          for _ in range(k_eff):
25              t = int(lg.reshape(-1, lg.shape[-1])[-1].argmax())
26              cands.append(t)
27              lg, cache = draft._run(
28                  draft.embed_tokens(torch.tensor([[t]], device=lg.device)),
29                  cache, need_logits=True)
30              if t == eos_id:
31                  break
32      return cands, cache, lg
```

输入

- `draft`：0.5B 模型；
- `cache`：**draft 的** cache，长度 `pre`；
- `lg`：draft 在位置 `pre-1` 的 logits，形状 `(1, 1, 151936)`；
- k_eff = min(k, 剩余预算)；
- `eos_id`：结束符 id（可为 None）。

逐行

- **第 22 行** `cands = []`：普通 Python `list[int]`，**故意不做成 tensor**。因为候选要被逐个喂回去自回归生成，攒成 list 才能一次打包给 target 做块前向。
- **第 25 行**：这是全文件最容易写错的一行。
- `lg.reshape(-1, lg.shape[-1])`：把 `lg` 无论 `(1,1,V)` 还是 `(1,V)` 都压成 `(n, V)`；
- `[-1]`：取最后一个位置的行 → `(V,)`；
- `.argmax()`：返回**标量张量**，`int()` 转成 Python int。
- **为什么不能写成 `lg[0,-1]`**：在 `(1,1,V)` 下它与 `lg[0,-1]` 得到的都是 `(V,)`，看起来等价；但一旦 `lg` 是 `(1,V)`，`lg[0,-1]` 取到的是**一个标量**，`argmax()` 恒返回 0 □ 候选永远是 token 0 □ 接受率恒为 0。我们真的踩过这个坑：接受率 0/58 却"结构检查全过"的**空过 PASS**（详见 6.4）。
- **第 26 行**：记录候选。
- **第 27–29 行**：`torch.tensor([[t]], device=lg.device)` 构造 `(1,1)` int64 张量（**device 跟随 logits**，因此 CPU/GPU 都能跑）；`embed_tokens` 查表 → `(1,1,hidden)`；`_run(x, cache, need_logits=True)` 内部：
- `start_pos = cache[0]["k"].shape[2]` —— **位置信息自动从 cache 长度推出**，RoPE 用的 `cos/sin` 自然落在正确位置上，不需要手传 position；
- 返回**新的 cache**（引擎内部是 `cache["k"] = torch.cat([...], dim=2)` 原地改 dict，再把同一批 dict 装进新 list 返回）；
- 返回 `lg (1,1,V)`：draft 对"下一个位置"的预测。
- **第 30–31 行**：draft 提议出 EOS 就停止提议。**注意顺序**：EOS 检查在"推进 cache"**之后**，所以 EOS 候选的 K/V 也会写进 cache，出口长度恒为 `pre + len(cands)`。顺序放反会造成全接受分支裁剪越界（详见 6.4 的 bug #2）。
- **第 32 行**：返回三元组 (候选 id 列表, 推进后的 draft cache, 推进后的 draft logits)。
- **契约**：返回的 `lg` 对应"`cands` 全部消费完之后"的位置（长度 `pre+kp`）—— 这正是"全接受"分支拿它当下一轮 `drf_lg` 的原因（第 102 行）。

3.5 `_verify`（第 35–43 行）—— target 一次判 k 个

```
35  def _verify(target, cache, cands):
36      with torch.inference_mode():
37          lg, cache_blk = target._run(
38              target.embed_tokens(
39                  torch.tensor([cands], device=cache[0]["k"].device)),
40              cache, need_logits=True)
41      kp = len(cands)
42      return lg[0, -kp:], cache_blk
```

- `torch.tensor([cands])` → `(1, kp)`；`embed_tokens` → `(1, kp, 1536)`；一次 `_run`，`start_pos = pre`。
- **因果 mask**（`s = kp > 1` 且 cache 非空）：

```
mask = torch.triu(torch.ones(s, T, device=..., dtype=torch.bool), diagonal=p + 1)   # p = T - s = pre
scores = scores.masked_fill(mask, float('-inf'))
```

含义：候选 `d_j` 只能看到位置 ≤ pre+j 的内容。**这是"一次喂 kp 个猜测"合法的根本原因** —— 位置 `pre+j` 的输出只依赖"它自己及之前"的 token，与我们逐个喂进去的结果在数学上等价（数值上只差计算形状带来的舍入噪声，第 6 节量化了这一点）。

- cache 写回：base 档 `torch.cat([cache["k"], k], dim=2)`；v_fp8 档 K 保持全长 bf16、V 存 FP8 payload + fp32 scale，读路径先反量化再拼接。
- **第 42 行** `lg[0, -kp:]`：`(1, kp, V)` → `(kp, V)`。**位置映射必须背下来**：

变量	来自	其 logits 所属位置	用来判定
`saved_next`	上一轮（或 prefill）末尾	`pre-1`	`cands[0]`
`rows[0]`	块前向第 0 行	`pre`	`cands[1]`
`rows[j]`	块前向第 j 行	`pre+j`	`cands[j+1]`
`rows[kp-1]`	块前向最后一行	`pre+kp-1`	**不判任何候选**

`rows[kp-1]` 是"假设 kp 个候选全对"时下一个位置 `pre+kp` 的 logits，全接受时被当作下一轮的 `saved_next` 复用 —— 这是"一次前向产出 kp+1 个位置信息"里被省下的那一次单步前向。

3.6 `_step`（第 45–52 行）—— 单 token 递推

```
45  def _step(model, token, cache):
46      with torch.inference_mode():
47          lg, cache = model._run(
48              model.embed_tokens(
49                  torch.tensor([[token]], device=cache[0]["k"].device)),
50              cache, need_logits=True)
51      return lg[:, -1:], cache
```

- cache 长度 L → L+1；返回 `lg[:, -1:]` = `(1, 1, V)`（**统一形状合同**，`_propose` 第 25 行的 reshape 正是为兼容这两种形状）。
- 两个用途：① 错配分支算替换 token `r` 的 logits（成为下一轮 `saved_next`）；② 把 `r` 喂给 draft。
- 同一函数用形参 `model` 服务两个模型，避免复制两份代码。

3.7 主循环（第 54–116 行）

签名

```
54  def spec_decode_greedy(target, draft, prompt_ids, max_new_tokens, *, k=K,
55                         eos_id=None, target_kv="base_bf16", draft_kv="base_bf16",
56                         log=None):
```

`prompt_ids` `(1,S)`；`max_new_tokens` **只算新生成段**（不含 prompt）；`target_kv/draft_kv` 是实验条件；`log` 是每轮回调（G5 记录、B6 归因都靠它）。

```
57      target.set_kv_format(_fmt(target_kv))
58      draft.set_kv_format(_fmt(draft_kv))
```

有状态地修改两个模型每一层的缓存格式 □ 两个档必须串行跑、每次调用都重设。`v_fp8` 会断言 `head_dim % 128 == 0`，所以它只能用于 1.5B target（0.5B 的 head_dim=64），draft 恒为 base。

```
60      tgt_lg, tgt_cache = _prefill(target, prompt_ids)
61      drf_lg, drf_cache = _prefill(draft, prompt_ids)
62      saved_next = tgt_lg[:, -1:]
63      drf_lg = drf_lg[:, -1:]
```

两个模型 prefill **完全相同的 prompt**（同 tokenizer、同模板）。否则两边看到的上下文不同，接受率就没有意义。第 62 行的 `saved_next` 是"判第一个候选的人"，第 63 行取 `(1,1,V)` 作为提议起点。

```
65      gen = []
66      st = dict(rounds=0, proposed=0, accepted=0, target_block_calls=0, draft_calls=0,
67                mismatch_rounds=0, all_accept_rounds=0, rollbacks=0)
69      while len(gen) < max_new_tokens:
70          k_eff = min(k, max_new_tokens - len(gen))
71          pre = spec_cache.cache_length(tgt_cache)
72
73          cands, drf_cache, prop_lg = _propose(draft, drf_cache, drf_lg, k_eff, eos_id)
74          kp = len(cands)
75          st["draft_calls"] += kp
76          if kp == 0:
77              break
79          rows, tgt_blk = _verify(target, tgt_cache, cands)
80          st["target_block_calls"] += 1
81          st["proposed"] += kp
```

- 第 70 行：末轮自动缩小提议数（`max_new_tokens=2` 时 `k_eff=2`；预算 1 时 `k_eff=1`）。
- 第 71 行：`pre` = 下一个要写的位置索引，也是**所有裁剪的基准**。
- 第 76–77 行：防御性检查（当前实现下不会为 0，因为循环至少跑一次）。

```
83          g = [int(x.argmax()) for x in [saved_next] + [rows[j] for j in range(kp - 1)]]
84          m = 0
85          while m < kp and g[m] == cands[m]:
86              m += 1
```

- 第 83 行构造 kp 个"判决者"：第 0 个来自上一轮遗留的 `saved_next`，其余来自块内 `rows[0..kp-2]`；**`rows[kp-1]` 不在列表里**。
- 第 85–86 行求**最长相等前缀** `m`。语义重点：这是**逐个精确相等**，不是"按概率比接受"。因为 greedy 下 target 的 argmax 就是唯一正确答案，不等就必须丢弃。

```
88      if m < kp:                    # 分支 A：错配
89              r = g[m]
90              emitted = cands[:m] + [r]
91              tgt_cache = spec_cache.trim_cache(tgt_blk, pre + m)
92              saved_next, tgt_cache = _step(target, r, tgt_cache)
93              drf_cache = spec_cache.trim_cache(drf_cache, pre + m)
94              drf_lg, drf_cache = _step(draft, r, drf_cache)
95              st["mismatch_rounds"] += 1
96              st["rollbacks"] += 1
97      else:                         # 分支 B：全接受
98              emitted = list(cands)
99              tgt_cache = tgt_blk
100             drf_cache = spec_cache.trim_cache(drf_cache, pre + kp)
101             saved_next = rows[kp - 1]
102             drf_lg = prop_lg
103             st["all_accept_rounds"] += 1
```

分支 A 的三个关键点

1. **第 89 行** `r = g[m]`：target 自己选的 token，它与 `cands[m]` 必然不同（`m` 是首个不相等的位置）。
2. **第 91 行为什么必须"裁掉"而不是"改写一格"**：`tgt_blk` 的长度是 `pre+kp`，其中位置 pre…pre+kp-1 的 K/V 属于**draft 猜的 token**，只有前 `m` 个是真的。裁到 `pre+m` 就把 d_{m+1}…d_{kp} 的 K/V 全部丢弃。

常见误区：有人会想"把位置 `pre+m` 的 K/V 直接换成 `r` 的就行"—— 错。那一格存的是**被拒绝的 `d_{m+1}`** 的 K/V（它是在 `d_m` 之后算出来的），而 `r` 需要的是"在 `d_m` 之后、`r` 自己之前"的位置。唯一正确做法就是裁到 `pre+m`，然后**重新前向一次 `r`**（第 92 行）。

3. **第 93–94 行**：draft 也必须走同一处理。两个模型的 cache 必须对应**同一条真实 token 序列**，否则下一轮 draft 的提议基于错误上下文，接受率会崩。`m=0` 时裁到 `pre`，等价于"整块回滚"。

分支 B：块前向写进 cache 的内容**已经全对**（那 kp 个位置就是这 kp 个真实 token），所以**不裁、不重算**（第 99 行）—— 这是投机解码真正省时间的地方。第 100 行是幂等的防御性裁剪；第 101 行把块内最后一行留作下一轮判决者。

```
105         gen += emitted
106         st["accepted"] += m
107         st["rounds"] += 1
108         if log is not None:
109             log(dict(round=st["rounds"] - 1, Kp=kp, m=m, pre=pre,
110                      tgt_len=spec_cache.cache_length(tgt_cache),
111                      drf_len=spec_cache.cache_length(drf_cache),
112                      cands=cands, g=g, emitted=emitted))
113         if emitted and emitted[-1] == eos_id:
114             break
116     return {"ids": list(prompt_ids[0]) + gen, "gen": gen, "stats": st}
```

- 第 106 行：`accepted` **只累加 m**。替换 token 是 target 自己产出的，不计入接受率的分子（但计入产出）。**报告里必须写清这个定义**，否则接受率不可比。
- 第 113 行用 `emitted[-1]` 判停是安全的：`_propose` 的 break 保证 EOS 只可能出现在 `cands` 末尾。
- 第 116 行：`ids` 是完整序列（prompt+gen），`gen` 只含新生成段 —— **B6 比较的就是 `gen`**。

3.8 统计字段字典

字段	含义	用途
`rounds`	轮数	——
`proposed`	Σ kp，一共提议了多少候选	接受率分母
`accepted`	Σ m，被 target 确认的候选数	接受率分子（**不含替换 token**）
`target_block_calls`	target 块前向次数（== rounds）	验证开销
`draft_calls`	draft 前向次数（== proposed）	draft 侧开销
`mismatch_rounds`	有候选被拒的轮数	回滚次数、每轮多产 1 个 target token
`all_accept_rounds`	全部接受的轮数	最理想路径占比
`rollbacks`	回滚次数（当前实现与 mismatch 恒等）	cache 裁剪开销

---

4. 引擎侧：cache 与量化到底在算什么

4.1 cache 的四种形态

fmt	K	V	说明
"fp16"（= base）	计算 dtype	计算 dtype	不做任何量化。**名字是历史遗留**
"v_fp8"（正式）	bf16 全长	FP8 E4M3 + fp32 scale	Asymmetric V-only，GROUP_V=128（1 head 1 组）
"fp8"（负消融）	FP8	FP8	全 FP8，粒度由 fp8_g_k/v 旋钮控制
None	——	——	首次 prefill，无 cache

fmt 的解析在 SelfAttention.forward（model_self.py 第 113–122 行）：

```
113  fmt = cache.get("fmt", self.kv_format) if cache is not None else self.kv_format
116  if fmt == 'v_fp8':
117      k_on, v_on, gk, gv = False, True, GROUP_SIZE, GROUP_V
118  elif fmt == 'fp8':
119      k_on, v_on, gk, gv = self.fp8_k, self.fp8_v, self.fp8_g_k, self.fp8_g_v
120  else:
121      k_on = v_on = False
```

4.2 量化算子逐行

```
67   E4M3_MAX = 448.0
68  GROUP_SIZE = 32     # full-FP8 ablation 默认组粒度
69  GROUP_V = 128       # 正式 Asymmetric V-only: V 的 group size(=1 group/head)

71   def quantize_kv_fp8(x: torch.Tensor, group_size: int = GROUP_SIZE):
72       d = x.shape[-1]
73       assert d % group_size == 0, f"head_dim {d} % group_size {group_size} != 0"
74       g = d // group_size
75       xg = x.to(torch.float32).view(*x.shape[:-1], g, group_size)
76       m = xg.abs().amax(dim=-1, keepdim=True)
77       s = m / E4M3_MAX
78       s = torch.where(s == 0, torch.ones_like(s), s)
79       q = (xg / s).to(torch.float8_e4m3fn)
80       q = q.reshape_as(x)
81       return q, s.squeeze(-1)

83   def dequantize_kv_fp8(q, scale, dtype=torch.float16):
84       g = scale.shape[-1]
85       qg = q.to(torch.float32).view(*q.shape[:-1], g, q.shape[-1] // g)
86       hat = qg * scale.unsqueeze(-1)
87       return hat.reshape_as(q).to(dtype)
```

逐行含义：

- **第 75 行** `x.to(torch.float32).view(..., g, group_size)`：先把输入升到 fp32（避免 bf16 求最大值时的精度损失），再把最后一维（head_dim=128）**切成 g 组。**GROUP_V=128 时 g=1，即整个 head 一组；GROUP_SIZE=32 时 g=4。
- **第 76 行** `xg.abs().amax(dim=-1, keepdim=True)`：**每组取绝对值最大** → scale 的分子。这是 "per-group absmax"（在线量化，不校准、不需要统计数据集）。
- **第 77–78 行** `s = m / 448`：E4M3 的可表示最大绝对值是 **448**，所以让"组内最大值"正好映射到 448。`torch.where(s == 0, 1, s)` 处理**全零组**（否则会出现 0/0 = NaN）。
- **第 79 行** `(xg / s).to(torch.float8_e4m3fn)`：归一化后**转成 FP8。**这一步是有损的：E4M3 只有 3 位尾数。

- **第 81 行** `s.squeeze(-1)`：把 `(..., g, 1)` 压成 `(..., g)`，作为"每组一个 scale"存下来。对 V（`GROUP_V=128`）来说就是 `(1, 2, T, 1)` —— **每个 token、每个 KV 头一个 scale。**
- **第 85–86 行（反量化）**：`q.to(torch.float32)` 升回 fp32，按同样的分组形状乘回 scale。注意反向 reshape 用的是 `q.shape[-1] // g`，即从 scale 的组数推出组大小 —— 因此**同一份 payload 必须用同一套 group 设置反量化。**
- **第 87 行** `.to(dtype)`：降回计算 dtype（bf16/fp32）后参与 attention。**我们不做 FP8 矩阵乘**，只是"存 FP8、算的时候反量化"。

质量代价（实测，G3 冻结）：V-only FP8 在 WikiText-2 的 40 个独立窗口上 ΔNLL = +0.000397、ΔPPL = +0.040%；而 K-FP8 的所有粒度都使 ΔNLL ≥ +2.4（不可用），full-FP8 更差。**这就是为什么正式方案是"Asymmetric V-only"。**

4.3 写路径与"当前块也量化"的语义

```
124  if k_on:
125      qk_cur, ks_cur = quantize_kv_fp8(k, gk)
126      k_cur16 = dequantize_kv_fp8(qk_cur, ks_cur, x.dtype)
129  if v_on:
130      qv_cur, vs_cur = quantize_kv_fp8(v, gv)
131      v_cur16 = dequantize_kv_fp8(qv_cur, vs_cur, x.dtype)
```

当前这一块的 K/V 也先量化再反量化（`k_cur16`/`v_cur16`），存进 cache 的 payload 与参与 attention 的那一份**完全相同。**这条语义很重要：

- 保证结果与 chunk 大小无关（分块 prefill 与一次性 prefill 数值等价，V3 已实测）；
- 避免"同一 token 在 cache 里和在当前块里精度不一致"这种隐蔽 bug。

存储与读取（第 136–155 行）分两种模式：`fp16` 档直接 `torch.cat` 到全长；FP8 档**按有无 `*_scale` 键判断该层是 payload 还是已反量化的全长历史**，读时先反量化再拼接：

```
144  if "k_scale" in cache:
145      k_all = torch.cat([dequantize_kv_fp8(cache["k"], cache["k_scale"], x.dtype), k_cur16], dim=2)
150  if "v_scale" in cache:
151      v_all = torch.cat([dequantize_kv_fp8(cache["v"], cache["v_scale"], x.dtype), v_cur16], dim=2)
```

对 v_fp8 而言 K 走 `else` 分支（无 `k_scale`），是全长的 bf16 历史并**原地写回**；V 则始终以 FP8 payload + scale 形式保存。

4.4 attention 里每个操作在算什么

```
156  k_rep = kv_repeat(k_all, self.heads // self.kv_heads)    # (1,2,T,d) → (1,12,T,d)
157  v_rep = kv_repeat(v_all, self.heads // self.kv_heads)
158  scores = torch.matmul(q, k_rep.transpose(-2, -1)) / (self.head_dim ** 0.5)
169  attn = torch.softmax(scores, dim=-1, dtype=torch.float32).to(q.dtype)
170  out = torch.matmul(attn, v_rep)
```

- `kv_repeat`：GQA 的关键 —— 用 `unsqueeze(2).expand(...)` 再 `reshape`，把 2 个 KV 头复制成 12 个 query 头（**不复制数据，`expand` 是视图**，`reshape` 时才按需物化）。
- `transpose(-2,-1)` + `matmul` → `scores (1, 12, S, T)`：query 与所有历史 K 的点积，除 `sqrt(head_dim)` 做缩放。
- `masked_fill(mask, -inf)`：把不该看的位置压成负无穷，softmax 后权重为 0。
- `softmax(..., dtype=torch.float32)`：**在 fp32 中做 softmax 再转回** —— 这是我们在 G1 对齐阶段定下的规则；bf16 直接 softmax 会放大误差。
- `matmul(attn, v_rep)` → 加权求和 → `transpose` → `view` 拼回 `(1,S,1536)` → `o_proj`。

另外两处 fp32 内部计算（同样来自 G1 的对齐经验）：

- `RMSNORM.forward`：`x32 = x.to(torch.float32)` → 求均方 → `rsqrt(variance+eps)` → 乘权重 → 转回原 dtype。bf16 下直接算方差会丢精度。
- `Build_Rope_cache` + `apply_rope`：`cos/sin` 用 fp32 生成，位置索引也是 fp32（`calc_dtype = torch.float32`），再 `to(dtype)` 后与 q/k 相乘。

为什么强调这些：本机 CUDA fp16 对该 Qwen 路径会产生 NaN（早期 PPL 出现 1e7 级异常），因此**正式质量与性能口径一律 BF16**，并把上述三处内部计算固定为 fp32。这些不是"调参"，是让数值可信的必要条件。

---

5. 逐行讲 `spec_cache.py`（用户实现，B4 PASS）

5.1 `cache_length`（第 4–7 行）

```
4   def cache_length(cache):
5       if cache is None:
6           return 0
7       return int(cache[0]["k"].shape[2])
```

- 输入：cache（list[dict] 或 None）。
- 输出：Python `int`，token 维长度。
- **None → 0 的设计理由：**让"还没有 cache"成为一种**合法状态**，而不是到处写 `if cache is None`。于是 trim_cache(None, 0) → None 也自然成立。
- **为什么读 `cache[0]` 就够了**：所有层的 token 长度必须一致（由 `check_consistent` 守卫），所以只看第 0 层是安全的；同时这也是"长度唯一来源"原则的一部分。

5.2 `trim_cache`（第 10–30 行）

```
10  def trim_cache(cache, new_len: int):
11      if cache is None:
12          if new_len == 0:
13              return None
14      raise ValueError("trim_cache(None,n > 0) 非法参数输入.")
15      cur = cache_length(cache)
16      if not isinstance(new_len, int) or isinstance(new_len, bool):
17      raise ValueError(f"new_len 必须是int.")
18      if new_len <= 0 or new_len > cur:
19      raise ValueError(f"new_len {new_len} 越界 (current {cur})")
21      out = []
22      for i, layer in enumerate(cache):
23          nl = {}
24          for key, val in layer.items():
25              if isinstance(val, torch.Tensor):
26                  nl[key] = val[:, :, :new_len]
27              else:
28                  nl[key] = val
29          out.append(nl)
30      return out
```

契约（B4 逐条实测通过）：

情形	行为	为什么
`cache is None, new_len == 0`	返回 `None`	允许"从空开始"
`cache is None, new_len > 0`	`ValueError`	无中生有是非法的
`new_len <= 0`	`ValueError`（**包括 0**）	见下
`new_len > cur`	`ValueError`	不能"补长"
`new_len` 非 int 或 bool	`ValueError`	防止浮点/布尔误传
正常	浅拷贝，**所有**张量沿 dim=2 裁	——

逐行要点

- **第 26 行** `val[:, :, :new_len]`：切片沿 **dim=2**（token 维）。因为 cache 里每个张量都是 `(1, kv_heads, T, head_dim)` 或 `(1, kv_heads, T, 1)`（v_scale）—— **这一行同时裁掉了 `v_scale`**，不会出现"V 裁了、scale 没裁"的错位。
- **第 25 行** `isinstance(val, torch.Tensor)`：只裁张量，**跳过非张量字段** —— `"fmt"` 是字符串（如 `"v_fp8"`），必须原样保留，否则下一次前向会因为读不到格式而走错分支。

- **第 21–30 行返回的是新 dict 组成的 list**：原 cache 的 dict 与张量**不被修改**（切片产生视图，但原张量本身不变）。B4 里专门测了 `orig-unchanged=True`。
- 为什么必须这样：`tgt_blk`（块前向的结果）在错配分支里还要被"裁"；如果 `trim` 原地改，就会污染 `tgt_blk` 本身，而全接受分支正需要它的原始长度。
- **第 18 行 `new_len <= 0` 判非法**（而不是允许裁到 0）：状态机里真实 token 数恒 `> 0`（至少包含 prompt）。**允许 0 会掩盖 `pre + m` 算错的 bug** —— 例如 `m=-1` 时裁到 0 会"成功"，于是错误被静默吞掉。这是"宁可响亮报错"的设计。

5.3 `check_consistent`（第 32–46 行）

```
32  def check_consistent(cache):
33      if cache is None:
34          return
35      lens = []
36      for i, layer in enumerate(cache):
37          cur = []
38          for key, val in layer.items():
39              if isinstance(val, torch.Tensor) and val.dim() >= 3:
40                  cur.append((key, int(val.shape[2])))
41          vals = {n for _, n in cur}
42          if len(vals) != 1:
43              raise RuntimeError(f"cache token dim mismatch at layer {i}: {cur}")
44          lens.append(cur[0][1])
45      if len(set(lens)) != 1:
46          raise RuntimeError(f"cross-layer token length mismatch: {lens}")
```

- **第 39 行 `val.dim() >= 3`**：只检查三维以上的张量（K/V/v_scale 都是 3 维以上），跳过标量类字段。
- 两级守卫：**层内**（同一层的 k/v/v_scale 长度必须一致）与**跨层**（各层长度必须一致）。B4 里人为把某一层的 v 裁短，守卫立刻抛 `RuntimeError`。
- 这是"长度唯一来源"原则的**执行者**：任何将来引入的新字段只要被裁错，这里就会炸。

5.4 为什么必须"裁剪"而不是"就地改一格"

这是全项目最容易理解错的一处，用数字说明。设 `pre=12`，候选 `d1..d4` 被写进位置 `12..15`：

- target 接受了 `d1`（位置 12）、拒绝了 `d2`，替换 token 是 `r`；
- **位置 13 的 K/V 是 `d2` 的**（它在计算时"看到"的是 `d1` 之后接 `d2`）；
- 而 `r` 需要的是"看到 `d1` 之后接 `r`"的 K/V —— 这两者不同；
- 所以位置 13 及其之后的所有 K/V 都不可用，**必须裁掉**（`trim_cache(tgt_blk, 12+1=13)`），再对 `r` 做一次单 token 前向，让它以正确的前缀写进位置 13。

裁剪的第二个作用是把**draft cache 拉回同一真实序列**：如果 draft 的 cache 里还留着被拒的 `d3`、`d4`，下一轮它的提议就基于错误上下文，接受率会崩。

5.5 自测脚本（第 49–65 行）

文件内自带 `if __name__ == "__main__"` 自测：验证 `cache_length`/`None` 语义、`trim` 后长度与"原 cache 未被修改"、非法长度抛错、坏 cache 被 `check_consistent` 抓到。这是 B4 正式测试 `g4_b4_cache_tests.py` 之外的快速自检。

---

6. 正确性是怎么被证明的（含一个真实反例）

6.1 判据

B6 门：对每个 case、每个 KV 档，

```
spec_decode_greedy(...)["gen"]  ==  target_only_greedy(ids, n)   # 逐 token id 比较
```

参照是**同一引擎的 cached AR 贪心生成**（不是 HF），另加：logits 全 finite、重复运行确定、`accepted > 0`（防空过）、case 与档位必须跑满。

6.2 两档并列登记

档	配置	结果
B6-fp32	CPU / float32 / batch=1 / k=4	**PASS：10 case × 2 KV 档 = 20/20 组合逐 token 全等**（总 accepted/proposed = 216/348）
B6-bf16	GPU / bf16（性能档）	**CHECK 19/20**：唯一分歧 `natural-2 @7`

为什么需要两档：**被测对象是实现**（控制流、cache 语义、logits 对齐、替换重算、EOS 边界），dtype 不是被测对象。而 bf16 在那一个位置上**无法表示这个决策**（见 6.3），所以把同一份实现放到决策良定义的配置（fp32，相对误差 ~1e-7，与 1e-1 量级的决策差距相差 5–6 个数量级）里测量。**这是"换仪器"而不是"放宽尺子"**：判据与 bf16 档逐字相同，且判据在运行前就已入库（`3f7fe6f`）。

6.3 那个"差一个 token"的真实案例

prompt：`"Photosynthesis is the process by which"`，分歧发生在生成第 7 个 token。在同一个 prefix（14 token）上，用 target 模型的多条计算路径取该位置 logits：

口径	`' use'` **(id 990)**	`' convert'` **(id 5508)**	gap	argmax
bf16 full / cached / AR-history	22.8750	22.8750	**0.0000**（精确并列）	三路径全 = `' use'`
fp32 full / cached / AR-history	23.1752	23.3243	**+0.1491**	三路径全 = `' convert'`

解读：

- bf16 在该位置把两个候选**压成了完全相同的值**（22.8750 = 该量级下 bf16 的 1 ULP 是 0.125，两者差 0）。
- fp32 给出真值：`' convert'` 高 **+0.1491**（≈1.2 个 bf16 ULP）。
- 同一位置的**形状噪声**实测为 full-vs-cached max|Δ| = 0.250、`hist-vs-full = 0.281`（≈2–4 ULP）。
- 于是 signal (0.1491) ≤ noise (0.25–0.28)：**该位置在 bf16 下数值不可判定**，与实现无关。
- 有趣的是：投机解码那一路恰好选中了 fp32 真值 `' convert'`（因为验证用的是块形状，亚 ULP 累积落在真值一侧），而 AR 参照选了按最小 id 破平局的 `' use'`。

这不是 bug，也不放宽门：台账里 bf16 档仍记 CHECK 19/20，该位置作为"已知数值限制"如实披露；同时在 fp32 档（良定义）逐 token 全等。附带一个对 **RQ2** 有用的观察：bf16 档接受率 50.9%（194/381），fp32 档 62.1%（216/348）—— bf16 的翻面**同时污染"输出 token"和"接受判定"**；不过这个差值属复合效应（两个模型的 argmax 都可能翻），单独归因需要专门的 tie 频率普查。

6.4 三个真实 bug 与修复（都写进了回归测试）

bug #1：logits 形状不一致导致"空过 PASS"。

早期主循环把 logits 保持为 `(1,V)`，而 `_propose` 用 `lg[0,-1]` 取标量 → `argmax` 恒为 0 → 候选永远是 token 0 → 接受率 0/58，但结构检查全部通过。修法：统一 logits 合同为 `(1,1,V)`（`_step` 返回 `lg[:,-1:]`），并把

_propose 改成 lg.reshape(-1, lg.shape[-1])[-1].argmax()；B6 增加 vacuity guard（accepted == 0 时 verdict 降为 CHECK）。

bug #2：EOS 候选被接受时 trim_cache 越界（生成正常收尾的路径）。

_propose 原先把 if t == eos_id: break 放在**推进 cache 之前**，于是当 draft 提议的最后一个候选是 EOS 时，draft cache 长度只有 pre+kp-1；当该轮被 target 全部确认（m == kp，即生成以 EOS 正常结束）时，全接受分支的 trim_cache(drf_cache, pre+kp) 因为 new_len > cur 抛 ValueError。修法：把 EOS 检查移到推进之后，使出口恒为 pre+kp。副作用是 draft_calls += kp 与实际 draft 前向次数**精确相等**（此前 EOS 轮少算 1 次）。

回归测试 g4_b5_eos_branch_tests.py 用**位置策略 stub 模型**在 CPU 上毫秒级复现整条分支（不需要权重/GPU/网络），并带**旧实现对照**：旧实现在 3 条 EOS 全接受 case 上如实抛 ValueError: new_len N 越界 (current N-1) —— 这保证测试不会"空过"。

bug #3：设备硬编码。

_verify 里写死 device="cuda"，导致 CPU/fp32 复算直接崩。修法：跟随 cache[0]["k"].device（与 _step、_propose 一致）。

6.5 一次真实生成的成本核算（RQ2 的量级感）

B5 smoke 的 16 token：accepted 11 / proposed 27（≈40.7%）⇒ **target 前向 13 次**（8 块 + 5 单步）对 AR 的 16 次，draft 前向 27 次。接受率是分子、单步重算与回滚是分母 —— **这就是 RQ2 要系统量化的权衡本体。**

---

7. 常见误解 Q&A

Q1：为什么不把 k 调大一点?

k 越大，一次块前向能覆盖的候选越多，但 draft 端要跑 k 次前向、块前向的算术量也按 k 增长，且接受率随位置迅速衰减（后面的候选要建立在前面全对的前提下）。协议把 k=4 冻结，是为了让"上下文长度"这个自变量干净。

Q2：为什么不给全接受轮一个 bonus token?

设计 v0.2/v0.3 明确**取消 bonus**。理由：bonus 会让"每轮产出"与"每轮前向次数"的对应关系变复杂，而我们研究的是接受收益 vs 验证开销的分解。现在全接受时 `rows[kp-1]` 只当"下一轮判决者"复用，不额外产 token。

Q3：`rows[kp-1]` 明明也是 target 算出来的，为什么不用它判 `cands[kp-1]`？

它的位置是 `pre+kp-1`，判的是**下一个**位置的 token（`cands[kp-1]` 之后的那个），用它判 `cands[kp-1]` 会整体错位一格。

Q4：错配时为什么不能让替换 token 复用块里的某一行 logits?

`rows[m]` 是"在 `d_{m+1}` 之后"的条件分布，而替换 token `r` 需要"在 `r` 自己之前"的上下文。两者上下文不同，复用等于用错误前缀做决策。

Q5：为什么两个模型的 cache 长度必须相等?

它们必须表示**同一条真实 token 序列**。draft 若保留了被拒 token 的 K/V，它下一轮的提议就基于错误历史，接受率会崩。

Q6：接受率怎么算？替换 token 算不算?

`accepted / proposed`，**替换 token 不计入分子**（它是 target 自己产出的，不是"被确认的候选"）。它计入产出，所以有恒等式 `gen_len == accepted + mismatch_rounds`。

Q7：greedy 版为什么不用概率比？采样版差在哪?

greedy 下 target 的答案唯一（argmax），所以"精确相等"就是完备判据。采样版中 target 的输出是分布抽样，必须按 `min(1, p_target(x)/p_draft(x))` 接受，并在拒绝时从修正分布 `norm(max(0, p_target - p_draft))` 重采样，才能保证输出分布与 target 单独采样一致。我们**未实现**采样版。

Q8：一次喂 k 个猜测，不会破坏因果性吗?

不会。因果 mask（`triu(..., diagonal=pre+1)`）保证位置 `pre+j` 的输出只能看到 ≤ pre+j 的内容，因此"第 j+1 个位置看到前面猜的东西"与"逐个喂"在数学上一致。数值上只差计算形状导致的舍入（实测 0.25–0.28，见 6.3）。

Q9：bf16 下偶尔差一个 token，算失败吗?

在**性能档**里它是"该位置数值不可判定"的表现，已量化归因并如实披露；在**良定义配置**（fp32）下同一份实现逐 token 全等。判据没有被放宽，两档并列登记。

Q10：KV 压缩（FP8）和投机解码怎么相互影响?

V-FP8 把 8K 逻辑 KV 从 224.00 MiB 压到 169.75 MiB（**1.320×**，每 token 每层 1024 B → 776 B），但读路径多了一次反量化。投机解码则减少 target 前向次数、从而减少 KV 读取次数。两者都作用在"每 token 要搬多少字节"这条线上 —— 这就是 RQ3（FP8 是否改变投机的有利上下文区间）的动机。

8. 速查表

8.1 形状与 dtype

对象	形状	dtype
`prompt_ids` / `ids`	`(1, S)`	int64
`embed_tokens(ids)`	`(1, S, 1536)`（target）/ `(1, S, 896)`（draft）	bf16 / fp32
`logits`	`(1, S, 151936)`	bf16 / fp32
`cache[layer]["k"]`	`(1, 2, T, 128)`（target；draft 为 `T,64`）	bf16 / fp32
`cache[layer]["v"]`（v_fp8）	`(1, 2, T, 128)`（target 专属格式）	float8_e4m3fn
`cache[layer]["v_scale"]`	`(1, 2, T, 1)`	fp32
`saved_next` / `_step` 返回	`(1, 1, V)`	bf16 / fp32
`rows`	`(kp, V)`	bf16 / fp32
`cands` / `g` / `emitted`	`list[int]`	——

8.2 关键不变量

1. `len(tgt_cache) == len(drf_cache) == prompt + len(gen)`（**已确认真实 token 数**）；
2. 每轮末：`tgt_len == drf_len == pre + (m+1 if m<kp else kp)`；
3. `_propose` 出口：`len(drf_cache) == pre + len(cands)`（EOS 也照常写入）；
4. 判决者对齐：cands[0] ← saved_next，cands[j] ← rows[j-1]，`rows[kp-1]` 不判；
5. `gen_len == accepted + mismatch_rounds`；
6. `target_block_calls == rounds, draft_calls == proposed`；
7. `T` 恒在 dim=2；`pre = cache[0]["k"].shape[2]` 是唯一长度来源。

8.3 验证命令

```
python -m specdec.engine.g4_b5_helper_tests        # T1-T4 辅助函数单测
python -m specdec.engine.g4_b5_smoke               # 主循环结构不变量 + 确定性
python -m specdec.engine.g4_b5_eos_branch_tests    # EOS 分支回归（纯逻辑 CPU，秒级）
python -m specdec.engine.g4_b6_correctness         # B6-bf16 exact-id 门（GPU）
python -m specdec.engine.g4_b6_fp32_correctness    # B6-fp32 实现正确性门（CPU/fp32）
python -m specdec.engine.g4_b6_tie_diag            # 分歧位置 signal-vs-noise 归因
```

8.4 一句话总结

投机解码用一个小模型**猜**、用大模型**一次批改**：大模型每轮只做一次块前向，就能确认若干候选并产出 target 自己也会产出的 token 序列；代码上它由 **`_propose`（猜）/ `_verify`（批改）/ 最长相等前缀（判）/ trim + 单步重算（回滚）**四件事组成，而全部正确性都压在一条不变量上**：两个 cache 的长度永远等于"已确认真实 token 数"**。